

Resumen del Ecosistema Hadoop

Introducción y Contexto

El ecosistema Hadoop extiende las capacidades básicas de Hadoop Core (HDFS, YARN, MapReduce) para transformarlo en una plataforma empresarial completa de Big Data. Mientras Hadoop Core proporciona almacenamiento distribuido y procesamiento por lotes, el ecosistema añade herramientas que facilitan la ingesta, procesamiento, análisis y administración de datos, haciéndolo accesible para diversos roles técnicos y de negocio.

Clasificación de Componentes del Ecosistema

1. Componentes de Acceso y Procesamiento de Datos

Estas herramientas simplifican el análisis de datos almacenados en HDFS, ofreciendo alternativas más accesibles que la programación directa en MapReduce.

- **Apache Pig:** Lenguaje de scripting de alto nivel (Pig Latin) que abstrae la complejidad de MapReduce. Permite definir flujos de transformación de datos de forma más intuitiva.
- Ejemplo: En lugar de escribir cientos de líneas de código Java en MapReduce para contar palabras, en Pig Latin se podría escribir: `words = LOAD 'texto.txt' AS (linea); palabras = FOREACH words GENERATE FLATTEN(TOKENIZE(linea)) AS palabra; agrupadas = GROUP palabras BY palabra; resultado = FOREACH agrupadas GENERATE group, COUNT(palabras);`
- **Apache Hive:** Sistema de almacenamiento y consulta que traduce instrucciones SQL-like (HiveQL) a trabajos MapReduce o Tez. Permite a usuarios familiarizados con SQL realizar consultas sobre grandes volúmenes de datos.
- Ejemplo: Para el análisis de efectividad de pases de jugadores, se podría usar una consulta como: `SELECT jugador, AVG(efectividad_pases) FROM estadisticas WHERE partidos_jugados > 20 AND ultimo_partido = TRUE GROUP BY jugador ORDER BY AVG(efectividad_pases) DESC;`

- **Apache Impala:** Motor de consultas SQL de alto rendimiento que ejecuta consultas directamente sobre datos en HDFS o HBase, sin traducirlas a MapReduce, ofreciendo latencias mucho menores (cercanas al tiempo real).
- **Apache Spark:** Framework de procesamiento unificado que supera las limitaciones de MapReduce. Permite procesamiento en memoria, mucho más rápido, y soporta lotes, streaming, SQL y aprendizaje automático. Es uno de los componentes más populares actualmente.
- **Apache HBase:** Base de datos NoSQL columnar distribuida que se ejecuta sobre HDFS. Está optimizada para lecturas/escrituras aleatorias y de baja latencia, a diferencia de HDFS que es secuencial.
- Caso de uso: Para consultar rápidamente las estadísticas de un jugador específico en un partido concreto, sin tener que escanear todo el histórico.
- **Apache Phoenix:** Capa SQL sobre HBase que permite ejecutar consultas SQL y compilarlas en escaneos nativos de HBase, combinando la facilidad de SQL con la velocidad de HBase.

2. Componentes de Ingesta y Flujos de Trabajo

Automatizan la captura de datos desde fuentes externas y la orquestación de pipelines o workflows.

- **Apache Sqoop:** Herramienta diseñada para transferir datos de forma bidireccional entre Hadoop y bases de datos relacionales estructuradas (como MySQL, Oracle).
- **Apache Flume:** Servicio distribuido para ingestar eficientemente grandes volúmenes de datos de streaming, como logs de servidores, directamente en HDFS.
- **Apache Oozie:** Servidor de flujos de trabajo que permite programar y orquestar tareas complejas de Hadoop (jobs MapReduce, Pig, Hive, Sqoop, etc.).

3. Interfaces y Herramientas de Trabajo

Proporcionan interfaces gráficas (GUI) y herramientas que mejoran la usabilidad y administración.

- **Apache Hue:** Interfaz web que agrupa aplicaciones para interactuar con Hadoop (editor de consultas Hive, Pig, navegador de HDFS, editor de Oozie workflows).

- **Apache Zeppelin:** Notebook web interactivo que permite realizar análisis de datos, visualización y colaboración usando lenguajes como SQL, Scala, Python.
- **Apache Ambari / Cloudera Manager:** Herramientas de provisión, administración, monitorización y seguridad de clústeres Hadoop.

4. Procesamiento en Streaming (Tiempo Real)

Permiten analizar datos en movimiento, a medida que se generan.

- **Apache Kafka:** Plataforma distribuida de streaming de eventos que funciona como un sistema de mensajería pub/sub de alta throughput. A menudo se usa como cola de ingesta para datos en tiempo real.
- **Apache Storm / Apache Flink:** Frameworks de procesamiento de streams distribuidos y tolerantes a fallos para analizar datos en tiempo real.
- Ejemplo práctico: Analizar los datos de los chalecos de los jugadores durante un partido para monitorizar fatiga o riesgo de lesión en tiempo real.

Roles en una Plataforma Big Data

- **Ingeniero de Datos (Data Engineer):** Construye y mantiene la infraestructura de datos (pipelines, ingestas, ETL).
- **Científico de Datos (Data Scientist):** Desarrolla modelos avanzados de IA y análisis predictivo.
- **Analista de Negocio (Business Analyst):** Resuelve preguntas de negocio usando herramientas de más alto nivel (como SQL o dashboards).

Conclusión

El ecosistema Hadoop no es monolítico; es un conjunto de proyectos open-source independientes que se integran para cubrir todas las fases del ciclo de vida de los datos. La elección de componentes depende de los casos de uso específicos (procesamiento por lotes vs. tiempo real, consultas SQL vs. procesamiento programático). Esta modularidad permite a organizaciones como el Real Ibérica Club construir una plataforma "data-driven" sin depender exclusivamente de desarrolladores expertos en MapReduce, acelerando la obtención de insights y la toma de decisiones basada en datos.

Resumen: Apache Pig y Apache Hive en el Ecosistema Hadoop

1. Apache Pig: Abstracción sobre MapReduce

Propósito: Simplificar la programación en Hadoop para analistas con conocimientos limitados de programación. - **Lenguaje:** Pig Latin (similar a SQL). - **Motor de ejecución:** Originalmente MapReduce, ahora también Tez o Spark. - **Ventaja:** Reduce drásticamente la cantidad de código necesario. Ejemplo: el conteo de palabras pasa de 62 líneas en Java a 7 en Pig Latin. - **Flujo de trabajo:** Describe transformaciones de datos (cargar, filtrar, agrupar, contar, ordenar, almacenar) que Pig traduce internamente en trabajos MapReduce. - **Características clave:** - Extensible mediante Funciones Definidas por el Usuario (UDF). - Ideal para procesos ETL (Extracción, Transformación y Carga). - Permite pruebas con subconjuntos de datos. - **Rendimiento:** Generalmente más lento que un programa MapReduce optimizado escrito a mano, pero el ahorro en tiempo de desarrollo lo compensa en muchos casos. - **Tendencia:** Está en desuso desde la llegada de Apache Spark.

2. Apache Hive: Data Warehouse sobre Hadoop

Propósito: Permitir que usuarios no técnicos (analistas, directivos) consulten datos en HDFS usando un lenguaje similar al SQL. - **Contexto:** Surgió en Facebook para superar las limitaciones de MapReduce y Pig, que aún requerían conocimientos de programación. - **Lenguaje:** HQL (Hive Query Language), un estándar de facto en Big Data muy similar a SQL. - **Arquitectura y Componentes:** 1. **Cliente:** Interfaces para conexión (JDBC, ODBC, Thrift), permitiendo integración con herramientas como Excel, Power BI o Tableau. 2. **Servicios (ejecutados en un nodo frontera):** - **HiveServer:** Gestiona peticiones concurrentes de clientes. - **Driver:** Orquesta la ejecución de una consulta. - **Metastore:** Almacena metadatos (estructura de tablas, ubicación de datos). - **Compilador:** Traduce consultas HQL a un plan de ejecución (DAG) para MapReduce, Tez o Spark. - **Optimizador:** Mejora la eficiencia del plan de ejecución. - **Características Principales:** - **Abstracción:** Define una estructura relacional (tablas) sobre ficheros en HDFS (CSV, ORC, Parquet, etc.). - **Accesibilidad:** Hace que Hadoop sea utilizable por analistas de negocio

gracias al lenguaje SQL estándar. - **Integración:** Conexión JDBC/ODBC permite a las herramientas de BI empresarial acceder a los datos. - **Extensibilidad:** Soporta UDFs para funcionalidades personalizadas. - **Seguridad:** Ofrece control de permisos a nivel de tabla y operación. - **Enfoque:** Optimizado para **consultas analíticas complejas** sobre grandes volúmenes de datos, no para operaciones transaccionales rápidas (OLTP). Una consulta de agregación masiva será más rápida que en una BD tradicional, pero una búsqueda de un solo registro será más lenta.

Conclusión General: Pig y Hive fueron respuestas evolutivas para hacer Hadoop más accesible y productivo. Pig automatizó la codificación MapReduce para desarrolladores, mientras que Hive democratizó el acceso a los datos para no programadores mediante SQL. Ambos comparten el principio de traducir una capa de abstracción de alto nivel (Pig Latin, HQL) en código de ejecución (MapReduce/Tez/Spark), sacrificando un posible rendimiento óptimo a cambio de una enorme ganancia en velocidad de desarrollo y accesibilidad.

Resumen Detallado: Hive - Arquitectura, HQL y Caso Práctico

1. Arquitectura de Ejecución en Hive

Hive convierte consultas HQL en tareas ejecutables en Hadoop mediante tres componentes clave: - **Compilador:** Traduce HQL a un plan de ejecución. - **Optimizador:** Mejora el plan para eficiencia y escalabilidad. - **Ejecutor:** Ejecuta el plan en Hadoop respetando dependencias, gestionando prioridades y recursos a través de YARN (Yet Another Resource Negotiator).

Ejemplo Práctico: El rendimiento de una consulta para calcular el promedio de ventas en una cadena de retail depende directamente de la capacidad de procesamiento del clúster Hadoop subyacente.

2. Acceso y Conectividad de Hive

Hive permite el acceso desde aplicaciones externas al clúster Hadoop. - **Mecanismos de Conexión:** ODBC, JDBC o Thrift. - **Caso de Uso:** Una aplicación

web en un servidor distinto puede consultar promedios de ventas por tienda conectándose a HiveServer mediante estos protocolos.

3. HiveQL (HQL) - El Lenguaje de Consulta

HQL es un lenguaje de alto nivel con sintaxis similar a SQL, diseñado para consultar datos almacenados en HDFS.

3.1. Modelo Conceptual

- **Base de Datos:** Agrupa tablas relacionadas (ej: `ventas` , `rrhh`). Facilita la gestión y los permisos.
- **Tabla:** Contiene datos de una entidad (ej: `ventas`). Se define sobre archivos existentes en HDFS.
- **Campo/Columna:** Propiedad de la tabla con un tipo de dato específico (ej: `importe` , `fecha`).
- **Registro/Fila:** Instancia individual de datos en la tabla.
- **Partición:** División lógica de una tabla para optimizar consultas. Los datos de cada partición se almacenan en directorios separados.
 - **Ejemplo:** Una tabla global de `ventas` se puede particionar por `tienda` y `año` . Una consulta para la tienda 1 en 2022 solo leerá los archivos de esa partición, no toda la tabla.

3.2. Tipos de Datos en Hive

- **Primitivos:** Enteros (`INT` , `BIGINT`), decimales (`FLOAT` , `DECIMAL`), fechas (`DATE` , `TIMESTAMP`), cadenas (`STRING` , `VARCHAR`), booleanos.
- **Compuestos:**
 - `ARRAY` : Lista de elementos del mismo tipo. Ej: `["toalla", "camisa"]` .
 - `MAP` : Colección de pares clave-valor. Ej: `{1121: "toalla", 3324: "camisa"}` .
 - `STRUCT` : Estructura con propiedades definidas. Ej: `{"IP": "127.0.0.1", "TipoCliente": "VIP"}` .

3.3. Tipos de Sentencias HQL

- **DDL (Data Definition Language):** Define y modifica estructuras.
- **DML (Data Manipulation Language):** Manipula y consulta datos.

Sentencias DDL Principales (Ejemplos)

- **Bases de Datos:** `sql CREATE DATABASE my_db; USE my_db; SHOW DATABASES;`
- **Tablas:** `sql CREATE TABLE inmuebles ( id INT, direccion STRING, provincia STRING ) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t' STORED AS TEXTFILE;`

`ALTER TABLE inmuebles ADD COLUMNS (pais STRING); DESCRIBE FORMATTED inmuebles; `````

Sentencias DML Principales (Ejemplos)

- **Consulta (SELECT):** `sql SELECT provincia, COUNT(*) FROM inmuebles WHERE superficie > 100 GROUP BY provincia ORDER BY superficie DESC;`
 - **Capacidades Avanzadas:** Subconsultas, `JOIN` entre tablas, y funciones definidas por el usuario (UDFs).
- **Carga de Datos:**
 - Desde HDFS: `LOAD DATA INPATH '/ruta/archivo.csv' INTO TABLE tabla;`
 - Desde una consulta: `INSERT INTO tabla SELECT ... FROM otra_tabla;`
- **Modificación y Borrado:** `sql UPDATE inmuebles SET provincia = 'A Coruña' WHERE provincia = 'La Coruña'; DELETE FROM inmuebles WHERE provincia = 'Madrid' AND superficie > 500;`

4. Ejercicio Resuelto: Análisis de Empresas

Contexto: Tabla `empresas` con campos: `nombre`, `facturación`, `empleados`, `beneficios`. **Objetivo:** Listar empresas con más de 1000 empleados, ordenadas por margen (beneficios/facturación) descendente.

Solución HQL:

```
SELECT nombre,  
       (beneficios / facturación) * 100 AS margen  
FROM empresas  
WHERE empleados > 1000  
ORDER BY margen DESC;
```

5. Caso Práctico: Limitaciones y el Surgimiento de Apache Impala

Escenario: El Real Ibérica Club usa Hive para análisis de datos deportivos (estadísticas de partidos y jugadores), integrado con cuadros de mando en PowerBI.

Problema Identificado: Latencia alta en consultas interactivas. Cuando el entrenador o el director deportivo filtran o actualizan datos en PowerBI, la respuesta de Hive es lenta porque cada consulta se traduce en un trabajo MapReduce con una sobrecarga significativa de inicio. **Conclusión:** Hive es excelente para procesamiento por lotes (batch) de grandes volúmenes, pero no está optimizado para **consultas interactivas de baja latencia** que requieren respuestas en segundos. Esta limitación lleva a la necesidad de explorar herramientas como **Apache Impala**, diseñada específicamente para consultas SQL interactivas y rápidas sobre Hadoop.

Optimización del Ecosistema Hadoop: De Hive a Spark - Guía

Contexto y Problema Inicial

El equipo del Real Ibérica Club enfrenta problemas de rendimiento con **Hive**, donde las consultas tardan varios minutos, afectando cuadros de mando interactivos y análisis empresariales que requieren respuestas en segundos. Esto lleva a evaluar alternativas como **Impala** y otras tecnologías del ecosistema Hadoop.

⚡ Apache Impala: Alternativa para Consultas Rápidas

Origen y Propósito

- **Desarrollado por Cloudera** (2012) como alternativa/complemento a Hive para consultas **OLAP (Online Analytical Processing)**.

- **Objetivo:** Ofrecer respuestas en segundos para consultas interactivas de Business Intelligence.
- **Donado a Apache Foundation** en 2015, convirtiéndose en proyecto top-level en 2017.

Características Clave

- **Lenguaje:** HQL (Hive Query Language) - mismo que Hive.
- **Interfaz:** ODBC compatible.
- **Almacenamiento:** Soporta HDFS y HBase con formatos como Parquet, Avro, RCFile.
- **Metadatos:** Usa el mismo **Hive Metastore**.
- **Seguridad:** Autenticación Kerberos y autorización con Sentry.

Ventaja de Rendimiento (Histórica)

- **Implementación en C++** (código de bajo nivel vs. Java de Hive).
-). - **Optimizaciones avanzadas** en ejecución de consultas. - **Ejemplo de uso:** Consultas complejas con agrupaciones, subconsultas y funciones analíticas.

Problemas de Arquitectura

- **Procesos fuera de YARN:** Los daemons de Impala (en C++) ejecutan en nodos worker compitiendo por recursos con YARN.
- **Falta de gestión unificada:** Sin monitorización centralizada, causando bloqueos y reducción de rendimiento.
- **Convergencia de Hive:** Con la integración de **Spark como motor de procesamiento**, Hive mejoró su rendimiento, reduciendo la brecha con Impala.

Estado Actual

- **Auge 2014-2020**, pero muchas empresas migran de vuelta a Hive por: 1. Menor diferencia de rendimiento. 2. Problemas de gestión de recursos. 3. Madurez de Hive on Spark.

Apache HBase: Base de Datos Operacional en Tiempo Real

Caso de Uso del Real Ibérica Club

Necesitan un sistema para: - **Ingesta en tiempo real** de estadísticas durante partidos. - **Consulta inmediata** del histórico + datos en vivo por jugador. - **Hive no es adecuado** para operaciones CRUD atómicas.

¿Por qué HBase?

HDFS tiene dos limitaciones para casos operacionales: 1. **Inmutabilidad**: Los bloques no se modifican, se reescriben. 2. **Bloques grandes** (~128 MB): Ineficiente para operaciones pequeñas.

HBase resuelve esto siendo una **base de datos NoSQL clave-valor** sobre HDFS con: - **Acceso aleatorio** a registros específicos. - **Operaciones CRUD** (Create, Read, Update, Delete) eficientes. - **Escalabilidad horizontal** mediante sharding.

Características Técnicas

- **Modelo columnar flexible**:
- **Familias de columnas** definidas al crear la tabla.
- **Columnas dinámicas** por registro.
- **Row-key** como identificador principal (diseño crítico).
- **Persistencia**: Datos como bytes en HDFS.
- **API simple** (no SQL): Thrift, Avro, REST.
- **Operaciones básicas**:

```
bash create 'tabla', 'familia_columnas' put 'tabla',  
'rowkey', 'familia:columna', 'valor' get 'tabla', 'rowkey' scan 'tabla'
```

Ejemplo Aplicado

Para estadísticas de fútbol en tiempo real: - **Tabla**: `estadisticas_partidos` - **Familia de columnas**: `metricas` - **Inserción por evento**:

```
bash put  
'estadisticas_partidos', 'JUG001_PART20231001', 'metricas:goles', '2'  
bash put  
'estadisticas_partidos', 'JUG001_PART20231001', 'metricas:asistencias', '1'
```

Apache Phoenix: SQL sobre HBase

Problema Identificado

HBase es excelente para ingesta pero **limitado para explotación**: - Solo consultas por `row-key` o `scan` completo. - Difícil para agregaciones, joins o cuadros de mando.

Solución: Phoenix

- **Capa SQL sobre HBase** mediante JDBC.
- **Traduce SQL a operaciones nativas** de HBase.
- **Mantiene el rendimiento** de HBase.

Capacidades SQL

- **Comandos soportados**: `SELECT` , `UPSERT` , `DELETE` , `CREATE TABLE` , `JOIN` , etc.
- **Ejemplo de consulta compleja**: `sql SELECT jugador, SUM(goles) FROM estadisticas WHERE temporada = '2023' GROUP BY jugador;`
- **Integración** con herramientas BI estándar vía ODBC/JDBC.

⚙ Apache Spark: Plataforma Unificada de Procesamiento

Caso de Uso Avanzado

El Real Ibérica Club quiere ir más allá de la analítica descriptiva: - **Modelos predictivos** con Machine Learning. - **Pronósticos** de rendimiento de jugadores. - **Predicción** de resultados de partidos.

¿Qué es Spark?

- **Plataforma opensource** líder en procesamiento distribuido.
- **Estándar de facto** para ETL y Machine Learning en Big Data.
- **Unifica** procesamiento **batch** y **streaming**.

Características Principales

1. **Sin almacenamiento propio:** Usa HDFS, S3, etc.
2. **Tolerancia a fallos:** Reejecución automática de tareas.
3. **Escalabilidad:** Paralelización masiva de operaciones.
4. **Librerías integradas:** - **Spark SQL:** Consultas SQL. - **MLlib:** Machine Learning. - **Spark Streaming:** Procesamiento en tiempo real. - **GraphX:** Análisis de grafos.

Ejemplo de Aplicación

```
# Carga de datos desde HDFS
datos = spark.read.parquet("hdfs:///estadisticas_futbol")

# Entrenamiento de modelo predictivo
from pyspark.ml.regression import LinearRegression
modelo = LinearRegression().fit(datos)

# Predicción de valor de jugadores
predicciones = modelo.transform(nuevos_datos)
```

Resumen Comparativo y Recomendaciones

Tecnología	Fortalezas	Casos de Uso Ideales	Limitaciones
Hive	Consultas complejas, maduro, integración con Spark	Data Warehousing, ETL batch	Lento para OLAP interactivo
Impala	Baja latencia para consultas interactivas	Dashboards en tiempo real, BI	Problemas de gestión de recursos
HBase	CRUD en tiempo real, escalabilidad horizontal	Sistemas operacionales, ingesta continua	API limitado, sin SQL nativo

Tecnología	Fortalezas	Casos de Uso Ideales	Limitaciones
Phoenix	SQL sobre HBase, fácil integración	Explotación analítica de datos operacionales	Capa adicional, overhead mínimo
Spark	Procesamiento unificado, ML, streaming	ETL complejo, modelos predictivos, análisis avanzado	Curva de aprendizaje, gestión de recursos

Conclusión para el Real Ibérica Club

1. **Para consultas interactivas:** Evaluar **Hive on Spark** vs. Impala considerando gestión de recursos.
2. **Para ingesta en tiempo real:** **HBase** es ideal para estadísticas durante partidos.
3. **Para explotación de datos HBase:** **Phoenix** permite SQL familiar.
4. **Para modelos predictivos:** **Spark MLlib** es la solución estándar del ecosistema.

Tendencia Actual

El ecosistema converge hacia **Hive + Spark** como stack principal, con **HBase/Phoenix** para casos operacionales específicos, mientras **Impala** pierde adopción debido a los avances de Hive y sus desafíos arquitectónicos.

Apache Spark - Guía Completa

Origen e Historia

Apache Spark nació como un proyecto de investigación en la Universidad de California, Berkeley, liderado por **Matei Zaharia**. Su objetivo principal era superar las limitaciones de **MapReduce**, especialmente su ineficiencia al escribir resultados intermedios en HDFS en cada paso del procesamiento.

Cronología clave: - **Investigación inicial:** Motor que aprovecha la memoria RAM de los nodos para acelerar la computación distribuida - **2013:** Donación del proyecto a la Apache Software Foundation - **Fundación de Databricks:** Matei Zaharia y otros 6 cofundadores crean esta empresa que ofrece Spark con soporte empresarial - **Actualidad:** Principal herramienta para ingeniería de datos de gran volumen

Recursos oficiales: - Página oficial de Apache Spark - Sitio web de Databricks

Arquitectura y Componentes

Entornos de Ejecución

Spark puede ejecutarse en tres modos diferentes:

1. **Sobre YARN** (más común) - En clústeres Hadoop
2. **Sobre Mesos** - Gestor de recursos más escalable para entornos híbridos
3. **Modo Stand-alone** - Para despliegues pequeños o máquina única

Componentes Principales

Core de Spark: - Conjunto de librerías para computación distribuida - El programador no gestiona concurrencia, recursos o sincronización - Soporta múltiples lenguajes: Scala (nativo), Java, Python, R, SQL

Librerías Especializadas: - **Spark SQL:** Procesamiento con lenguaje SQL - **Structured Streaming:** Procesamiento en tiempo real - **MLlib:** Machine Learning - **GraphX:** Procesamiento de grafos - **SparkR:** Conexión con lenguaje R

Arquitectura de Ejecución

Driver → Gestor del Cluster → Ejecutores

Driver: - Ejecuta la función main - Mantiene el estado de la aplicación - Planifica y distribuye trabajo - Responde a entradas del usuario

Ejecutores: - Ejecutan tareas asignadas por el Driver - Informan del estado de ejecución - Se ejecutan en diferentes nodos del clúster

Componentes Detallados

Spark Core y RDDs

RDD (Resilient Distributed Dataset): - Abstracción principal de datos - Colección de elementos procesables en paralelo - **Immutable:** No se modifican, se crean copias

Tipos de Operaciones:

Transformaciones (ejecución diferida, generalmente en paralelo): - `map(func)` : Aplica función a cada elemento - `flatMap(func)` : Puede generar múltiples elementos por entrada - `filter(func)` : Filtra elementos según condición - `distinct()` : Elimina duplicados - `union(rdd)` : Une dos RDDs

Acciones (ejecución inmediata, NO siempre en paralelo): - `count()` : Cuenta elementos - `reduce(func)` : Agrega elementos - `take(n)` : Toma primeros n elementos - `collect()` : Recoge todos los elementos al Driver - `saveAsTextFile(path)` : Escribe en almacenamiento

Ejemplo de limitación de paralelización: Una ordenación global requiere procesamiento en un único nodo, no se puede paralelizar completamente.

Spark SQL

Ventaja principal: Mayor productividad usando SQL vs. programación con RDDs

Abstracciones: - **DataFrame:** Tablas con filas y columnas (RDD con esquema) - **DataSet:** Similar a DataFrame pero con tipos definidos

Fuentes de datos soportadas: - JSON, Parquet, Avro, CSV - Bases de datos: MySQL, PostgreSQL, HBase, Cassandra, Elasticsearch - HDFS, Amazon S3, Amazon RedShift - Conexiones JDBC

GraphX

Propósito: Procesamiento de grafos en paralelo

Estructura: Grafos dirigidos con vértices y aristas - **Ejemplo de uso:** Cálculo de rutas más cortas - **Operaciones:** `subgraph` , `joinVertices` , `mapReduceTriplets`

MLLib

Funcionalidades para Machine Learning: - **Algoritmos:** Clasificación, clustering, regresión, filtros colaborativos - **Obtención de features:** Extracción, transformación, reducción dimensional - **Generación de modelos:** Entrenamiento, evaluación, aplicación - **Persistencia:** Almacenamiento y carga de modelos - **Utilidades:** Álgebra lineal, estadística

⚖ Ventajas y Desventajas

Ventajas

1. **Velocidad superior:** Uso de memoria RAM vs. MapReduce que usa disco
2. **Unificación de paradigmas:** Batch y streaming con mismo modelo
3. **Integración amplia:** Compatible con HDFS, Hive, etc.
4. **API rica:** Procesamiento, ML, grafos en una plataforma
5. **Menor código:** Comparado con MapReduce
6. **Comunidad activa:** +1000 desarrolladores contribuyendo

Desventajas

1. **Perfil técnico requerido:** Necesita programadores especializados
2. **Dificultad de optimización:** Depuración compleja en producción
3. **Cambios frecuentes:** Evolución rápida que dificulta mantenimiento
4. **Recursos humanos:** Perfiles con salarios altos y rotación elevada

Componentes de Ingesta: Apache Sqoop

Propósito: Transferencia de datos entre Hadoop y repositorios relacionales

Funcionamiento: Traduce comandos a programas MapReduce

Comandos principales:

```
# Importar tabla a Hive
sqoop import --connect jdbc:mysql://db.foo.com/corp --table EMPLOYEES
```



```
# Exportar de HDFS a base de datos
sqoop export --connect jdbc:mysql://db.example.com/foo --table bar --
```

Modelo de funcionamiento: Cada nodo del clúster abre conexión directa con la base de datos, por lo que es crucial verificar la capacidad de la base de datos para manejar múltiples conexiones simultáneas.

Aplicación Práctica: Caso Real Ibérica Club

Contexto: Club deportivo implementando plataforma Hadoop + Spark

Reto: Ingesta organizada y desatendida de múltiples fuentes de datos - Bases de datos internas (socios, ventas, marketing) - Fuentes de datos externas públicas - Necesidad de herramientas como Sqoop para automatización

Solución: Uso del ecosistema Hadoop completo para crear pipeline de datos integral.

Resumen Detallado: Herramientas del Ecosistema Hadoop

1. Apache Sqoop (Herramienta de Ingesta de Datos)

- **Propósito:** Transferir datos entre bases de datos relacionales (como MySQL, Oracle) y Hadoop (HDFS, Hive, HBase). Es una herramienta de línea de comandos.
- **Operaciones Principales:**
 - **Importar:** Trae datos desde una base de datos relacional a Hadoop.
 - **Exportar:** Envía datos desde Hadoop a una base de datos relacional.

- **Consideraciones:** Permite gestionar el límite de conexiones concurrentes a la base de datos fuente. Aunque en 2021 se consideró que llegaba al final de su vida útil, sigue siendo ampliamente utilizado por su simplicidad.

2. Apache Flume (Herramienta de Ingesta de Datos en Streaming)

- **Propósito:** Servicio distribuido y confiable para recolectar, agregar y mover grandes volúmenes de datos de **streaming** o logs (ej: logs de sistemas, redes sociales, IoT, emails) hacia repositorios como HDFS.
- **Arquitectura (Flujo de datos):**
 1. **Source (Fuente):** Consume datos de una fuente externa (ej: Twitter, Kafka, archivo de log) y los coloca en uno o más canales.
 2. **Channel (Canal):** Almacena los eventos de forma transitoria hasta que son consumidos por el Sink. Ofrece persistencia (ej: en memoria, en archivo).
 3. **Sink (Sumidero):** Recoge los eventos del canal, los procesa y los escribe en un destino final (ej: HDFS, HBase).
- **Características:** Garantiza la entrega de datos y permite construir topologías complejas (ej: múltiples fuentes, canales paralelos, flujos encadenados).

3. Apache Oozie (Orquestador de Flujos de Trabajo)

- **Propósito:** Orquestar y programar **workflows** (flujos de trabajo) complejos y dependientes dentro del ecosistema Hadoop. Es clave para la automatización de procesos, como las ingestas nocturnas y las transformaciones posteriores.
- **Componentes de un Workflow:**
 - **Condiciones de inicio:** Temporal (ej: a las 2:00 AM) o basada en datos (ej: cuando llegue un archivo nuevo).
 - **Acciones:** Tareas a ejecutar (MapReduce, Hive, Pig, Spark, Sqoop, Shell, envío de email).
 - **Mecanismos de control:** Bifurcaciones, decisiones y uniones para definir la secuencia lógica.

- **Funcionamiento:** Los workflows se definen en XML (hPDL), son parametrizables y se gestionan a través de una interfaz web. Una acción solo se ejecuta cuando la anterior ha finalizado correctamente.

4. Interfaces y Herramientas de Trabajo

4.1. Hue (Interfaz Web de Usuario)

- **Propósito:** Interfaz web **amigable y gráfica** que facilita el uso de Hadoop a usuarios técnicos y de negocio, eliminando la necesidad de usar la consola de comandos.
- **Funcionalidades Clave:**
 - Editor SQL para Hive/Impala y otras bases de datos.
 - Explorador de HDFS y tablas de Hive/HBase.
 - Visualización de resultados de consultas y creación de dashboards.
 - Editor y monitor de flujos de Oozie

* Sistema de seguridad con autenticación de usuario. * **Público objetivo:** Usuarios con o sin conocimientos técnicos profundos.

4.2. Apache Zeppelin (Plataforma de Notebooks)

- **Propósito:** Herramienta web basada en **notebooks** para análisis de datos interactivo, colaborativo y reproducible. Ideal para Data Scientists.
- **Características:**
 - Soporta múltiples intérpretes/lenguajes: Spark, Python, R, Scala, SQL (Hive), Shell.
 - Permite crear narrativas con código, texto, gráficos interactivos y visualizaciones dinámicas.
 - Facilita la colaboración y el compartir notebooks, que pueden publicarse como páginas web independientes.
- **Diferencia con Hue:** Mientras Hue es una interfaz generalista y sencilla, Zeppelin es un entorno de desarrollo y exploración dirigido a profesionales que necesitan programar análisis complejos.

4.3. Herramientas de Administración del Cluster

- **Apache Ambari:**

- Herramienta **open-source** para la provisión, gestión, monitoreo y seguridad de un cluster Hadoop.
- Permite instalar servicios, configurarlos, iniciarlos/detenerlos y visualizar métricas de salud.

- **Cloudera Manager:**

- Ofrece funcionalidades **similares a Ambari**, pero es una herramienta **propietaria** de Cloudera (no es código abierto).

- **Importancia:** Ambas son esenciales para la gestión práctica de un cluster distribuido, haciendo manejable la administración de múltiples servicios y nodos.

5. Procesamiento en Streaming (Mencionado)

El texto introduce que para el procesamiento de datos en tiempo real (streaming), el ecosistema Hadoop ofrece frameworks como: * **Apache Spark (Structured Streaming)** * **Apache Flink** * **Apache Storm** Estas herramientas están diseñadas para procesar flujos continuos de datos con baja latencia.

Procesamiento de Datos en Tiempo Real en el Ecosistema Hadoop - Guía

Contexto y Necesidad del Real Ibérica Club

El **Real Ibérica Club** tiene una plataforma Hadoop madura para **Big Data** (datos en reposo), pero busca evolucionar hacia **Fast Data** (datos en movimiento) para procesar estadísticas y vídeos en tiempo real. Esto permitiría obtener más valor, como análisis instantáneos durante los partidos.

Big Data vs. Fast Data: Conceptos Clave

- **Big Data:** Se centra en **volumen**. Datos en reposo procesados de forma masiva en ventanas temporales (ej., cada noche). Tecnologías típicas: HDFS, Hive, Spark (en modo batch).
- **Fast Data (Streaming):** Se centra en **velocidad**. Datos generados continuamente ("gota a gota") y procesados en tiempo real. Ofrece un valor superior al permitir acciones inmediatas. Ejemplo práctico: Una fábrica que analiza sensores por lotes (Big Data) predice fallos para el día siguiente. Si los analiza en streaming (Fast Data), puede parar una máquina al instante al detectar una anomalía, evitando daños mayores.

⚙️ Conceptos Fundamentales para el Procesamiento en Streaming

1. Garantía de Procesamiento

Define la fiabilidad con la que se manejan los eventos. - **Exactly-once (Exactamente una vez):** Garantía de que cada evento se procesa **una sola vez**, incluso ante fallos. **Es crucial para transacciones financieras.** Ejemplo: Procesar un pago con tarjeta. Si se duplica, se cobraría dos veces al cliente. - **At-least-once (Al menos una vez):** Garantiza que todos los eventos se procesen, pero **podrían duplicarse** en caso de fallo. **Aceptable para métricas o dashboards.** Ejemplo: Actualizar un cuadro de mando con la temperatura de un sensor. Que un valor se procese dos veces no tiene un impacto crítico.

2. Modelo de Procesamiento

- **Evento a evento (One-at-a-time):** Procesa cada evento individualmente al llegar. **Ventaja:** Latencia mínima (milisegundos). **Desventaja:** Throughput (ancho de banda) potencialmente menor.
- **Micro-batch:** Agrupa eventos en ventanas cortas (ej., 1 segundo) y los procesa juntos. **Ventaja:** Throughput muy alto. **Desventaja:** Mayor latencia (la de la ventana).

Herramientas Principales de Streaming en Hadoop

1. Apache Storm

- **Modelo:** Procesamiento evento a evento.
- **Latencia:** Muy baja (milisegundos).
- **Garantía:** Nativa **at-least-once**. Para lograr exactly-once necesita **Trident** (aumenta la latencia).
- **Fortalezas:** Escalable, elástico, ideal para casos que requieren **respuesta ultra-rápida**.
- **Arquitectura:** Se programan **Spouts** (lectores de datos) y **Bolts** (unidades de procesamiento) que forman una **topología**. Ejemplo de topología: Para validar pagos con tarjeta, un Spout lee las transacciones, un Bolt las valida, y luego se bifurca: un flujo actualiza un dashboard y otro detecta fraude.

2. Apache Spark (Structured Streaming)

- **Modelo:** Procesamiento por micro-batches.
- **Latencia:** Baja, pero no en milisegundos (depende del intervalo del batch, ej., 1-2 segundos).
- **Garantía:** Exactly-once.
- **Fortaleza principal:** Reutilización de código y lógica con Spark Batch. Alto **throughput**. Ejemplo clave: El mismo código que valida pagos en streaming puede usarse por la noche para reprocesar (batch) todos los pagos del día con datos históricos adicionales.

3. Apache Flink

- **Modelo:** Procesamiento evento a evento por defecto, pero también soporta batch.
- **Latencia:** Muy baja (milisegundos).
- **Garantía:** Exactly-once.
- **Fortalezas:** Combina **baja latencia** con **alto throughput**. Especialmente bueno para **procesamiento de eventos complejos (CEP)** que requieren mantener estado.

- **Lenguajes:** Java, Scala, Python, SQL.

Recomendación: ¿Qué herramienta elegir?

La elección depende **estrictamente de los requisitos del caso de uso**:

1. **¿Latencia en milisegundos es crítica?** (Ej.: detección de fraude en <200ms).
 - Elegir entre **Apache Storm** o **Apache Flink**.
 - **Storm** si la prioridad es simplicidad y latencia extrema, aceptando at-least-once.
 - **Flink** si además se necesita exactly-once y alto throughput.
2. **¿La latencia de unos segundos es aceptable y se prioriza la integración con el ecosistema existente?**
 - Elegir **Apache Spark (Structured Streaming)**.
 - Especialmente recomendable si **ya se usa Spark para batch**, para reutilizar código, conocimiento y simplificar la arquitectura.

Para el Real Ibérica Club: Si su prioridad es analizar vídeo y estadísticas con la mínima demora, **Flink o Storm** serían óptimos. Si ya usan Spark para modelos predictivos y la latencia de unos segundos es aceptable, **Spark Structured Streaming** sería la opción más coherente y eficiente.

Autoevaluación (Respuestas Clave)

- Los casos de uso en tiempo real se adoptaron **después** de los de Big Data. **Verdadero.**
- En la garantía at-least-once, un evento podría **no procesarse**. **Falso.** Se garantiza que se procese al menos una vez (puede duplicarse).
- El procesamiento one-at-a-time es **siempre mejor** que micro-batch. **Falso.** Depende del caso de uso. Micro-batch ofrece mayor throughput, que es vital para volúmenes muy altos si la latencia no es crítica.